

Encontro 3: Motor de Busca Linear e Atualização de Dados

Desenvolvimento do MVP MecâniQA & Arquitetura de Memória Estática

CURSO
Sistemas de Informação

AUTORES
Gustavo Gabriel Gomes Mauricio e Equipe

LOCAL & DATA
Feira de Santana, 2026

☰ Agenda do Encontro 3



1. Assinatura da Busca Linear

Definição rigorosa dos parâmetros de entrada, tipos de retorno e convenção do código de erro para registros inexistentes.



2. Operação de Atualização (Update)

Mapeamento do fluxo de edição de registros mantendo a imutabilidade do código identificador único.



3. Reuso e Princípio DRY

Integração dos métodos estáticos para encadear a busca dentro do update, eliminando laços redundantes.



4. Análise do Limite de Memória

Defesa oral das restrições de arrays fixos (100 peças), estouro de capacidade e impacto no negócio.

</> Busca Linear: Parâmetros e Retorno



Parâmetros de Entrada (OAT)

- ✓ **Array Contíguo:** Referência para o vetor de dados (`Peca[]` ou `Servico[]`).
- ✓ **Tamanho Ativo:** `tamanhoAtual` (evita varrer posições nulas desnecessárias).
- ✓ **Chave de Busca:** `codigo` inteiro buscado.



Retorno & Convenção de Erro

- **Sucesso:** Retorna o índice de `0` a `N-1` .
- **Erro / Não Encontrado:** Retorna o inteiro `-1` .

Complexidade Temporal: $O(n)$

```
public static int buscarPecaPorCodigo(Peca[] array, int tamanhoAtual, int codigo) {  
    for (int i = 0; i < tamanhoAtual; i++) {  
        if (array[i] != null && array[i].codigo == codigo) return i; // Sucesso  
    }  
    return -1; // Indicativo de erro (Não encontrado)  
}
```


Operação de Update & Princípio DRY

Encadeamento de Métodos (DRY)

Para evitar duplicar laços `for`, o método `atualizarPeca` chama diretamente o motor de busca linear existente:

- **Passo 1:** Invoca `buscarPecaPorCodigo( ... )`.
- **Passo 2:** Se retornar `-1`, interrompe com erro.
- **Passo 3:** Se retornar `$\ge 0$`, acessa a posição direta no vetor.

Imutabilidade do Identificador

Garantia de integridade dos dados cobrada pelo negócio:

- 🛡 O `codigo` do registro permanece alterado **NUNCA**.
- ✎ Apenas nome, fabricante, custos e estoque são sobrescritos no objeto referente.

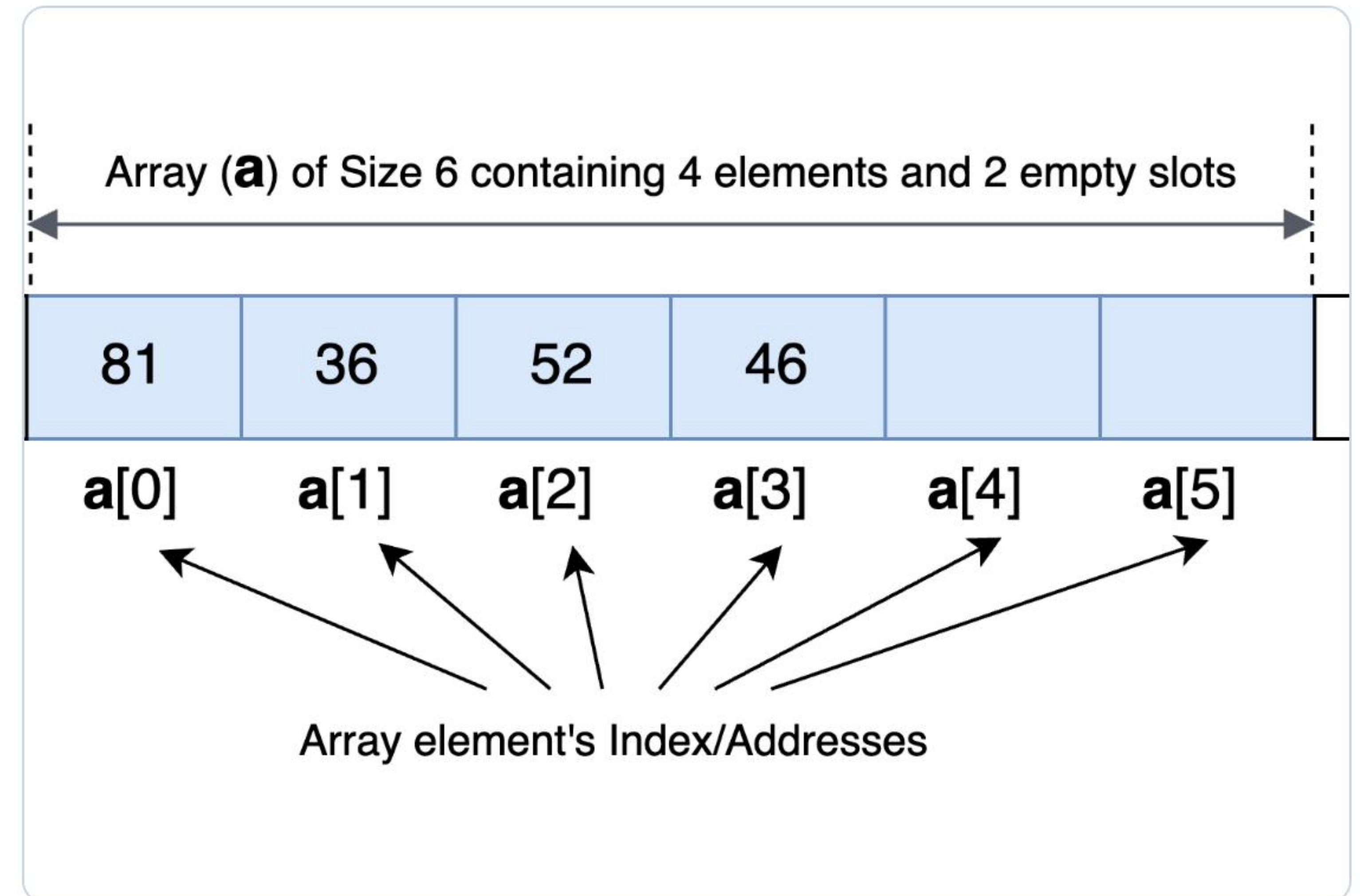
```
int indice = buscarPecaPorCodigo(pecas, totalPecas, codigo);
if (indice == -1) return false; // Aborta se não existir
Peca p = pecas[indice];
p.precoVenda = novoPrecoVenda; // Atualização direta dos atributos
```


⚠ Defesa do Limite Físico do Array

100

Limite Rígido de Peças no Array Estático

- ⚠ **Tratamento do 101º Item:** O método intercepta a validação `totalPecas ≥ MAX_PECAS` e bloqueia a operação, evitando a exceção fatal `ArrayIndexOutOfBoundsException`.
- 🏠 **Gargalo de Memória:** Espaço fixo reservado no Heap de imediato, podendo gerar desperdício se o uso for baixo.
- 📄 **Impacto no Negócio:** Impossibilita expansão sem refatoração do código-fonte.



Considerações Finais

Ganhos Teóricos & Práticos

- ✓ Algoritmo de busca sequencial simples e eficiente para volumes pequenos ($O(n)$).
- ✓ Manutenibilidade elevada ao concentrar a lógica de localização no `Gerenciador`.
- ✓ Código padronizado de acordo com o modelo de Pair Programming da equipe.

Próximas Evoluções (Futuras Sprints)

- > Substituição de arrays fixos por estruturas de dados dinâmicas (Listas Encadeadas / Listas Arranjo).
- > Implementação de algoritmos de ordenação para viabilizar Busca Binária ($O(\log n)$).

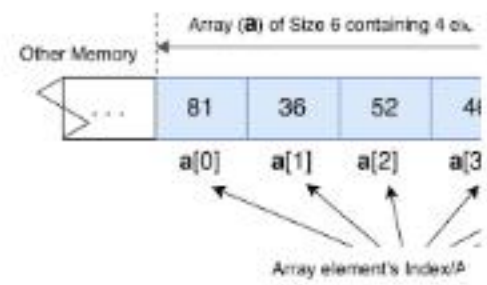
 DEFESA ORAL & AVALIAÇÃO

Dúvidas & Debate Técnico

Estamos prontos para responder às arguições sobre a arquitetura do MVP da MecâniQA e demonstrar os testes no `Main.java`.

`</>` Pair Programming: Dev (Code) & Colega (Logic Check)

Image Sources



<https://youcademy.org/array-data-structure/array-structure.png>

Source: youcademy.org